

 **sbintuitions/sarashina-embedding-v2-1b** 

 like

10

 Follow

 SB Intuitions

214

 Feature Extraction

 sentence-transformers

 Safetensors

 Transformers

 Japanese

llama

sentence-similarity

 text-generation-inference

 Model card

 Files and versions

 xet

 Community



 Train

 Deploy

 Use this model

Sarashina-Embedding-v2-1B

[日本語のREADME/Japanese README](#)

"Sarashina-Embedding-v2-1B" is a Japanese text embedding model, based on the Japanese LLM "[Sarashina2.2-1B](#)". We trained this model with multi-stage contrastive learning. We achieved the state-of-the-art average score across 28 datasets in [JMTEB](#) (Japanese Massive Text Embedding Benchmark). (Benchmarked on July 28, 2025.)

This model maps sentences & paragraphs to a 1792-dimensional dense vector space and can be used for semantic textual similarity, semantic search, paraphrase mining, text classification, clustering, and other applications.

Model Details

Model Description

- Model Type:** Sentence Transformer
- Base model:** [Sarashina2.2-1B](#)
- Maximum Sequence Length:** 8,192 tokens
- Output Dimensionality:** 1,792 dimensions
- Similarity Function:** Cosine Similarity
- Language:** Japanese
- License:** [Sarashina Model NonCommercial License Agreement](#)

Full Model Architecture

```
SentenceTransformer(  
    (0): Transformer({'max_seq_length': 8192, 'do_lower_case': False}) with  
    Transformer model: LlamaModel  
    (1): Pooling({'word_embedding_dimension': 1792, 'pooling_mode_cls_token':  
False, 'pooling_mode_mean_tokens': False, 'pooling_mode_max_tokens': False,  
'pooling_mode_mean_sqrt_len_tokens': False, 'pooling_mode_weightedmean_tokens':  
False, 'pooling_mode_lasttoken': True, 'include_prompt': False})  
)
```

Usage

First install the [Sentence Transformers](#) library:

```
pip install sentence-transformers==4.0.2
```

Then you can load this model and run inference.

```
from sentence_transformers import SentenceTransformer  
  
# Download from the 🤗 Hub  
model = SentenceTransformer("sbintuitions/sarashina-embedding-v2-1b")  
# Run inference  
query = [  
    'task: クエリを与えるので、与えられたWeb検索クエリに答える関連文章を検索してくださ  
い。 \nquery: Sarashinaのテキスト埋め込みモデルはありますか?'  
]  
texts = [  
    'text: 更級日記は、平安時代中期に菅原孝標女によって書かれた回想録です。',  
    'text: Sarashinaは、SB Intuitionsが開発した日本語大規模言語モデルです。これまでに  
7B, 13B, 70B, 8x70Bのモデルが公開されています。',  
    'text: サラシナエンベディングは日本語言語モデルをベースにした日本語埋め込みモデルで  
す。'  
]  
query_embedding = model.encode(query)  
text_embeddings = model.encode(texts)  
# Get the similarity scores between the embeddings  
similarities = model.similarity(query_embedding, text_embeddings)  
print(similarities)  
# tensor([[0.7403, 0.8651, 0.8775]])
```

How to add instructions and prefixes

For both the query and document sides, use different prefix formats. On the query side, add the prefix `task:` followed by instructions. (Only for STS task, both sentences are considered as query, and should be prefixed with the same instruction.)

- Query Side: `task: {Instruction}\nquery: {Query}`
- Document Side: `text: {Document}`

Templates for instructions and prefixes

The table below provides instruction and prefix templates for five main tasks.

Task	Query Side	Document Side
Retrieval Reranking	task: 質問を与えるので、その質問に答えるのに役立つ関連文書を検索してください。 \nquery:	text:
Clustering	task: 与えられたドキュメントのトピックまたはテーマを特定してください。 \nquery:	-
Classification	task: 与えられたレビューを適切な評価カテゴリーに分類してください。 \nquery:	-
STS	task: クエリを与えるので、もっともクエリに意味が似ている一節を探してください。 \nquery:	task: クエリを与えるので、もっともクエリに意味が似ている一節を探してください。 \nquery:

Training

Sarashina-Embedding-v2-1B is created through the following three-stage learning process:

Stage 1: Weakly-supervised Learning

To build a general-purpose and high-performance embedding model for a wide range of domains, we employed contrastive learning using weak supervision data, which consists of our own web-crawled data and open datasets.

Step2: Supervised Fine-tuning

To further train the model to better understand the similarity between queries and documents, we performed fine-tuning using higher-quality data than that used in Stage 1. Additionally, we trained multiple models by modifying parts of the data.

Stage 3: Model Merging

To enhance performance, we merged the weights of the two models that yielded the highest JMTEB scores in Stage 2 through linear merging.

Evaluation Results (*) with [JMTEB](#)

Model	Avg.	Retrieval	STS	Classification	Reranking	Clustering
Sarashina-Embedding-v2-1B (This model)	76.38	76.48	84.22	77.14	86.28	52.56
clnagoya/ruri-v3-310m	75.85	76.03	81.59	77.65	85.84	50.52
sbintuitions/sarashina-embedding-v1-1b	74.87	74.53	81.71	77.20	84.36	50.30
OpenAI/text-embedding-3-large	73.86	71.95	82.52	77.27	83.06	51.82

(*) Evaluated on July 28, 2025.

License

This model is licensed under [Sarashina Model NonCommercial License Agreement](#).

If you are interested in using this model for commercial purposes, please feel free to contact us through our [contact page](#).

[Edit model card](#)



 Safetensors 

Model size

1.22B params

Tensor type

 Files info

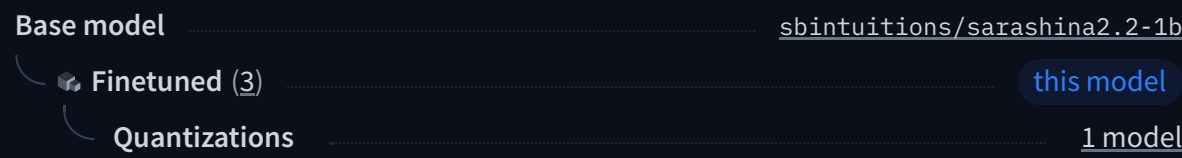
Inference Providers NEW

 Feature Extraction

This model isn't deployed by any Inference Provider.

 Ask for provider support

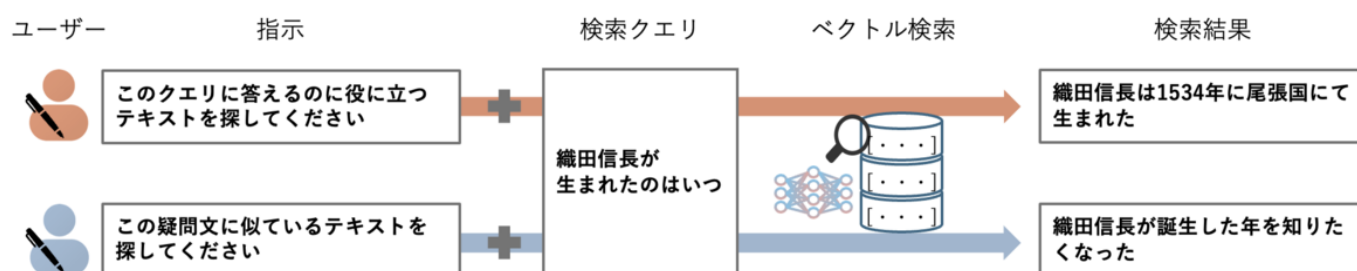
Model tree for sbintuitions/sarashina-embedding-v2-1b



2025-08-20

Sarashina-Embedding-v2-1B: 日本語に特化した指示を付与できるテキスト埋め込みモデル

Embeddings



TL;DR

- [sarashina2.2-1B](#)をベースに対照学習したテキスト埋め込みモデルを構築
- クエリに対して指示文で付与することで、タスク毎にテキストベクトルを調整可能
- JMTEBベンチマークで最高水準のスコアを達成(2025年7月28日時点)
- 特にRetrieval、STS、Rerankingタスクでハイスコア
- 公開ページ: <https://huggingface.co/sbintuitions/sarashina-embedding-v2-1b>

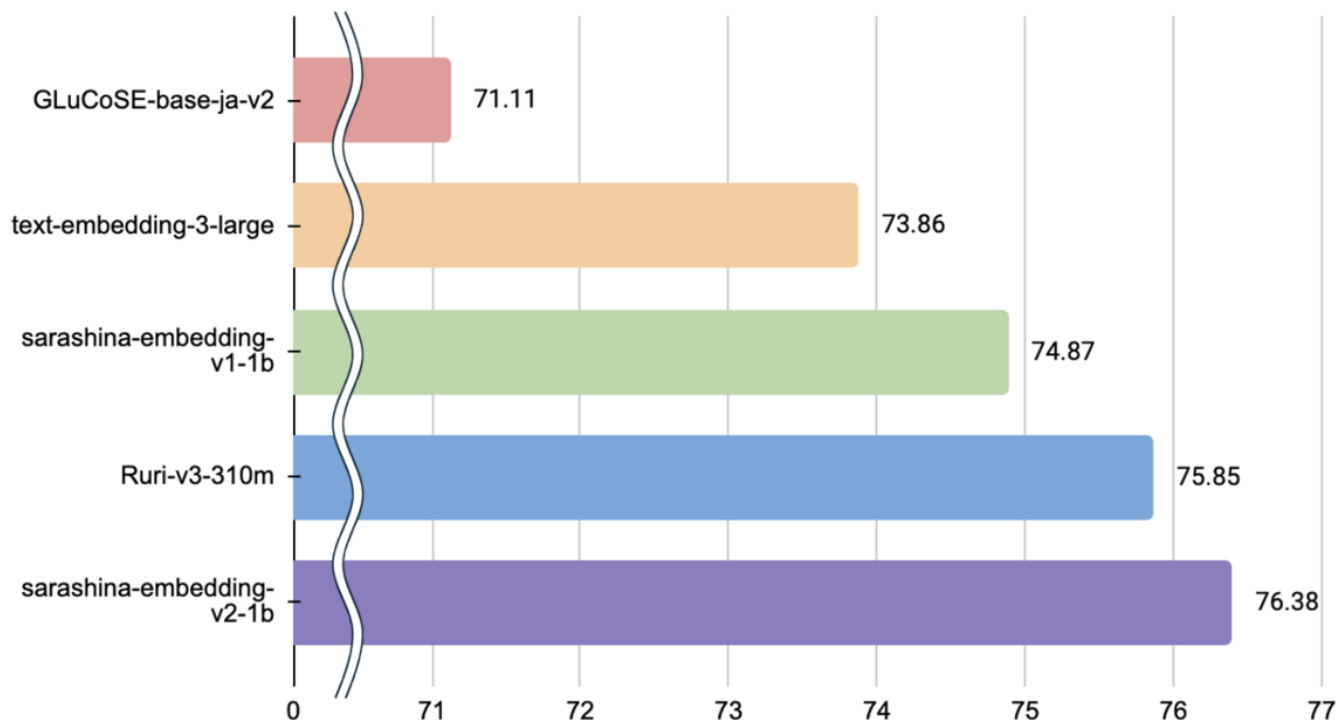


図1: JMTEBを用いた既存モデルとSarashina-Embedding-v2-1Bの性能比較

概要

こんにちは、SB Intuitions株式会でインターンをしている小川です。本ブログでは日本語特化の指示を付与できるテキスト埋め込みモデルである**Sarashina-Embedding-v2-1B**について紹介します。はじめに本モデルの特徴となる**指示**について実例を交えつつ紹介し、その後学習手法と評価結果についても説明します。

指示を付与できるテキスト埋め込みとは

テキスト埋め込みモデルとはテキストをベクトルに変換するモデルです。このベクトルは「意味の近さ」を反映し、類似の文（例:「俺は福岡生まれ福岡育ちです」と「私が生まれ育ったのは九州北部です」）は近い位置に配置されます。テキストの意味をベクトルに変換することで、情報検索や質問応答システム、テキストクラustering、文書分類といったさまざまな用途で使用できます。"テキスト埋め込みモデル"についてもっと詳しく知りたい方は「[Sarashina2.1-1BのTech Blog その1](#)」をご参照ください。

今回構築したSarashina-Embedding-v2-1Bの最大の特徴は、使用者が用途に合わせた指示をクエリに付与できる点です。これにより、指示に沿った最適なクエリのテキストベクトルを得られます。例えば「クエリを与えるので、もっともクエリに意味が似ている一節を探してください。」という指示をクエリに付与してモデルに入力することで、意味的類似性(STS)タスクで使用することを想定したベクトルを得ることができます。英語圏をメインターゲットとしている[Qwen3 Embedding](#)や[SFR-Embedding-Mistral](#)などのモデルはこういった指示を付与することができるモデルとして構築されており、テキスト埋め込みベンチマークである[MTEB](#)において高スコアを記録しています。

使い方

この節では、どのようにSarashina-Embedding-v2-1Bを使えば良いかについて解説していきます。前節でも述べたように、本モデルは使用用途に応じた指示をクエリに付与して使用します。そのために、クエリ側と検索対象側それぞれに次に示すような異なる形式のプレフィックスと指示を付与します。

クエリ側: `task: {instrcution}\nquery: {クエリ}`

検索対象側: `text: {検索対象のテキスト}`

例えば、Retrievalタスクでは次のような指示を付与します。

クエリ側: `task: クエリを与えるので、与えられたWeb検索クエリに答える関連文章を検索してください。\nquery: 更級日記を書いたのは誰ですか?`

検索対象側: `text: 更級日記は、平安時代中期に菅原孝標女によって書かれた回想録です。`

STSタスクは「似ている文を探したい」というタスクなので、両方のテキストをクエリとみなし次に示すように両方のテキストに同じ形式のプレフィックスと指示を付与します。

クエリ側: `task: クエリを与えるので、もっともクエリに意味が似ている一節を探してください。\nquery: Sarashinaは、SB Intuitionsが開発した日本語大規模言語モデルです。これまでに7B, 13B, 70B, 8x70Bのモデルが公開されています。`

検索対象側: `task: クエリを与えるので、もっともクエリに意味が似ている一節を探してください。\nquery: サラシナエンベディングは日本語言語モデルをベースにした日本語埋め込みモデルです。`

その他の主要タスクに付与するinstructionの例については[モデルのREADME](#)をご参照ください。

指示の影響

では、指示が検索にどのような影響を与えるのでしょうか。本ブログでは指示による検索結果の変化を「**織田信長が生まれたのはいつ**」というクエリを用いて検証してみました。下図のように異なる2つの指示(指示A,指示B)の下で4つの検索候補の検索時の並びがどのように変化するかを比較します。

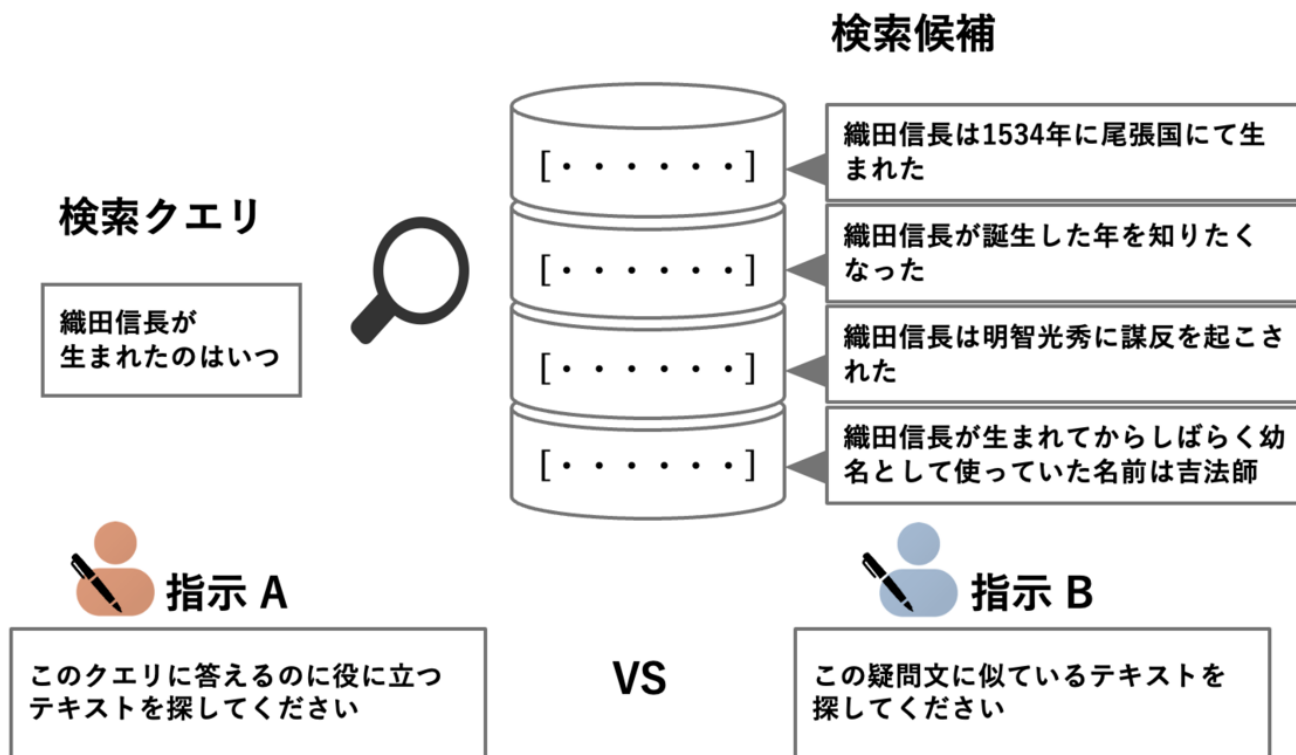


図2: 検証に使用する検索の設定

まず、指示A「このクエリに答えるのに役に立つテキストを探してください」を使用した場合、検索結果のランキングは以下のようになります。

順位	テキスト
1	織田信長は1534年に尾張国にて生まれた
2	織田信長が誕生した年を知りたくなった
3	織田信長が生まれてからしばらく幼名として使っていた名前は吉法師
4	織田信長は明智光秀に謀反を起こされた

この場合、クエリに直接答える情報を含む「**織田信長は1534年に尾張国にて生まれた**」が検索最上位に位置しています。この指示では、クエリに対する直接的な回答を提供するドキュメントが優先的に選ばれることが示されています。

次に、指示B「この疑問文に似ているテキストを探してください」(*)を使用した場合、検索結果のランキングは以下のようになります。

順位	テキスト
1	織田信長が誕生した年を知りたくなった
2	織田信長は1534年に尾張国にて生まれた
3	織田信長が生まれてからしばらく幼名として使っていた名前は吉法師

順位	テキスト
4	織田信長は明智光秀に謀反を起こされた

(*):プレフィックスの形式はSTS

この場合、クエリと意味的に類似している「**織田信長が誕生した年を知りたくなった**」が検索最上位に位置しています。この指示では、クエリに対する直接的な回答よりも、意味的に近いテキストが優先されることが示されています。

このように、指示によって検索ランキングが変化することが確認できます。

学習手法

この節では、Sarashina-Embedding-v2-1Bの学習手法について紹介します。Sarashina-Embedding-v2-1Bは下図のような3段階に分かれた学習ステップで構築されています。

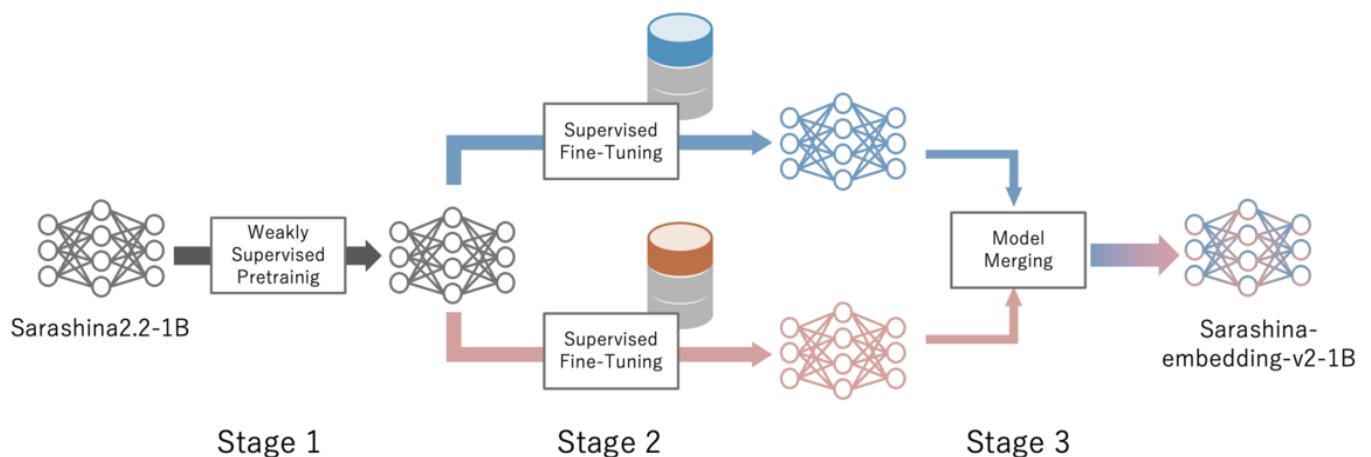


図3: Sarashina-Embedding-v2-1Bの構築の流れ

学習手法については本ブログでは手短に紹介します。

"弱教師あり学習,教師あり学習,対照学習"についてもっと詳しく知りたい方は「[Sarashina2.1-1BのTech Blog その2](#)」をご参照ください。

Stage 1: 弱教師あり学習

はじめに、弱教師あり学習(weakly-supervised-learning、WSL)によりモデルを対照学習で訓練します。このステージでは、ある程度関連性や類似性があるテキストのペアを用いて大規模な学習データで対照学習を行いました。

Stage 2: 教師あり学習

このステージでは、ステージ 1と比較すると小規模な学習データではあるものの、より高品質な学習データを用いて対照学習を行いました。学習データは"クエリ-正例-負例"の三つ組で構成されます。また学習データには全て指示とプレフィックスを付与して学習しています。そして学習データの一部を変更し複数のモデルを構築しました。

また、このステージでは次に挙げるような工夫をしています。

- 1. Synthetic data: LLMを用いてテキストを元にクエリとそれに対する指示を合成しました。(LLMを用いて合成したデータ以外の学習データには各タスクごとに使用できる汎用的な指示を付与しています。)
- 2. LLM-based Hard Negative mining: 負例として一見関連はしていそうだがクエリに対して不正解であるテキストを用意する必要があります。そのようなテキストを見つける工程をHard Negative miningと呼びます。一般的にはrerankerと呼ばれる小さなモデルで行いますが1.で合成したデータに対してはよりパラメータの多いLLMを用いたHard Negative miningで学習データを構築しました。
- 3. Example-based approach: 分類、クラスタリングタスク用の学習データは"テキスト-正解ラベル-不正解ラベル"の三つ組ではなく同一のラベルがついているテキストを正例とし、異なるラベルがついているテキストを負例に用いた"テキスト-テキスト-テキスト"の形式で構築しています。テキストとラベルの組み合わせで学習するよりも性能が向上することが[先行研究](#)によって示されています。

Stage 3: モデルマージ

このステージでは、ステージ 2で構築した学習データの一部が異なる複数のモデルのうちJMTEBの平均スコアが高かったモデルAとモデルBの重みの平均を取ることで性能向上を図りました。

性能評価

3つのステージを経て構築されたSarashina-Embedding-v2-1Bの性能を、5タスク・28データセットで構成されている日本語版テキスト埋め込みベンチマークであるJMTEBで計測しました。

JMTEBについての詳細は弊社ブログ「[日本語テキスト埋め込みベンチマークJMTEBの構築](#)」をご覧ください。

計測結果(*)を以下の表に示します。

Model	Avg.	Retrieval	STS	Classification	Reranking	Clustering
Sarashina-Embedding-v2-1B(This model)	76.38	76.48	84.22	77.14	86.28	52.56
Ruri-v3-310m	75.85	76.03	81.59	77.65	85.84	50.52
Sarashina-Embedding-v1-1B	74.87	74.53	81.71	77.20	84.36	50.30
text-embedding-3-large	73.86	71.95	82.52	77.27	83.06	51.82

(*) Evaluated on July 28, 2025.

Sarashina-Embedding-v2-1Bは、JMTEBの28データセットの平均スコアで**76.38**であり最高水準のスコアを達成しました。特に、既存のモデルと比較してRetrievalタスク、STSタスク、Rerankingタスクにおける性能が高いことが確認できます。

おわりに

本ブログでは、日本語特化の指示を付与できるテキスト埋め込みモデルである**Sarashina-Embedding-v2-1B**を紹介しました。JMTEBにおいて最高水準のスコアを達成した本モデルは、テキスト埋め込みモデルとして非常に高い性能を示しています。また、指示の付与を可能にしたことで、同一のクエリであっても用途に応じたベクトルに変換をし幅広いテキストを柔軟に扱えるようになりました。

今後の日本語テキスト埋め込みモデルの研究の方向性として、テキストだけでなく画像や音声などのマルチモーダルな情報への対応が挙げられます。これら多様なモダリティの情報とテキストを組み合わせることで、より高精度な検索やRAG(検索拡張生成)が可能になると期待できます。そのため、日本語に強いマルチモーダル埋め込み表現の研究も取り組む価値があるのではないかと考えています。

ライセンス

このモデルは、[Sarashina Model NonCommercial License Agreement](#)に基づいて公開されており、商用利用には制限があります。本稿をご覧いただいた方の中で、事業やプロジェクトに Sarashina-Embedding-v2-1B を使ってみたいという場合、お気軽にSB Intuitions にお問い合わせください。<[コンタクトページ](#)>

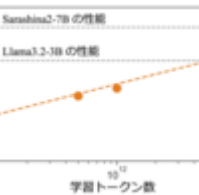
sraym 53日前



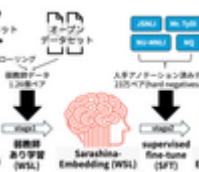
関連記事



2025-03-17
Sarashina2-Vision: 日本語特化の大規模視覚言語モデルの公開
概要 SB Intuitionsでは、日本語や日本の文化・慣習に強い大規...

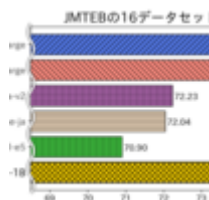


2025-03-06
Sarashina2.2: 数学・コーディングタスクの性能を向上させた日本語言語モデル
概要 Pretrainingチームの高瀬翔と李凌寒です。弊チームでは日...

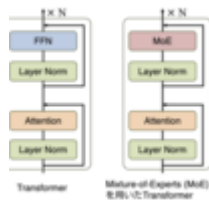


2025-02-03
Sarashina-Embedding-v1-1B: 日本語LLMをベースにしたテキスト埋め込み(2...
基本編はこちら→Sarashina-Embedding-v1-1B: 日本語LLMをベース...

2025-01-15



Sarashina-Embedding-v1-1B: 日本語LLMをベースにしたテキスト埋め込み(1...
TL;DR Sarashina-Embedding-v1-1Bという日本語特化テキスト埋め...



2024-11-25
Sarashina2-8x70Bの公開
概要 SB IntuitionsのSeng Pei Liew、李凌寒、高瀬翔です。弊...



コメントを書く

«『JamC-QA』: 日本の文化や風習
に特化した... ICML 2025に論文が採択されまし
た »

Copyright © SB Intuitions Corp. All right reserved.